

TrAI_NER: Prototyping a NER-centered custom NLP tool with pragmatic querying.

Emanuele Lionello

¹ *Language Technologies and Digital Humanities, 1062983*

Abstract

This paper details the AI-assisted development process of a local based software/web app that tries to mix Named Entity Recognition (NER, using SpaCy) and pragmatic analysis using LLMs, to find sentences with specific meaning inside a corpus. The idea for this software emerged both for personal curiosity and the need in past projects of NER inside bigger and more powerful tools like Sketch Engine, that almost don't provide this feature. Mixed results emerged from this attempt, because, as expected, the purely statistical function of SpaCy returned discrete results, even if not perfect, while the pragmatic analysis with LLM is still unusable, at least with the limited resources I had access to. The main problem was that, paradoxically, LLMs can't count due to their own statistical nature.

Keywords

Large Language Models, Named Entity Recognition, querying corpora.

1. Introduction

The aim of TrAI_NER was to try and understand two main limitations about modern and powerful tools like Sketch Engine (Kilgarrriff et al., 2004) and Antconc (Anthony, 2022): why they don't support NER natively and why they don't integrate LLMs in their querying capabilities. So, I decided to try and create my own basic corpus query engine to test and find out. First, it needs to be clarified that both Sketch Engine and Antconc don't support NER for primarily "philosophical" reasons, because they are intended more for research usage than for developing (like SpaCy) and NER is not broadly considered essential for these applications.

To make an example for a case in which this tool could be useful, I will briefly explain a past research I tried to make that has been heavily limited by the lack of NER in the Sketch Engine: the aim of the research was to examine in the ItTenTen family corpora whether there have been a reduction in the use of the male as norm in Italian over time, specifically regarding stereotypically high vs low status careers (e.g. "*Maria è ingegnere*", is technically wrong because the right form would be "*ingegnera*"). The problem is that querying only with CQL was not sufficient, because of one main reason: it is possible to filter for proper nouns and then for feminine tags e.g. `[tag="NPFS.*"][]{0,4}[lemma="essere|diventare|lavorare"][]{0,4}[lemma="ingegnere"]` within `<s>`, but results were heavily corrupted with a high amount false positives regarding organizations, and since there was no NER filter, there was no way to clean the data enough for it to be usable. Introducing the ability to filter for person would have fixed the problem automatically, and implementing NER inside these tools would allow for more complex and faster analysis.

2. Structure of the tool

2.1 The Components

The software has been built using Python on top on several different libraries: SpaCy (Honnibal & Montani, 2017) for the analysis itself, Streamlit (Streamlit Inc., 2024) and IO for the web UI, Ollama (Ollama, 2024) for importing open source LLM models, Pandas (McKinney, 2010), Matplotlib (Hunter, 2007), Numpy (Harris et al., 2020), Wordcloud (Mueller, 2023) and Seaborn (Wascom, 2021) for data representation, and BeautifulSoup (Richardson, 2024) for XML parsing (also Counter from collections and RE for regular expression matching for basic logic inside the script). The local models used are three and all available from the open source Ollama platform: Llama3 (AI@Meta, 2024) 8B parameters, Deepseek-r1:8b (DeepSeek-AI, 2025) and, most importantly, Qwen2.5-14B (Qwen Team, 2024). It was useful to test each of these three models available to find the most adequate for the job: Llama3 is small and fast with 8 billions parameters, Deepseek-r1 also has 8 billions parameters but it differs from the former because

it's specifically built for reasoning, while Qwen2.5 has 14 billions parameters, so it was helpful to understand whether scaling with bigger models returned better results. Another option would be to just use API for accessing much bigger and powerful models. For the sake of methodological transparency, it must be noted that the application was built utilizing an *AI-assisted development* paradigm (also known as LLM-driven rapid prototyping). While the architectural design, the selection of the NLP pipelines, the definition of the lexical metrics, and the prompt engineering rules were entirely human-led, the material generation of the Python syntax was facilitated by interacting with Large Language Models.

2.2 The User Interface (UI)

I structured the UI in different main areas: on top the Statistical section with graphical data visualization, on the left a menu with general and NER-specific statistics about the corpus used and then, scrolling down the dashboard, the pragmatic analysis section. In the first statistical section the normalization base of frequencies is automatically adjusted based on the number of tokens, if the corpus is less than 1000 tokens, the normalization base is going to be 100, if the corpus is under 100.000 tokens, it's going to be 10.000, and so on (Image 2. Graphical quantitative statistics section. In Appendix). In the Pragmatic analysis section, the use of NER-filtered results from SpaCy allows the LLM to significantly reduce the scope of the analysis, so that it significantly improves precision and reduces hallucinations and loading time, in fact the model does not have access to the whole corpus, but just on the results that are showed (Image 5. In Appendix). One of the first problems that emerged from SpaCy's analysis has been the long time needed for loading the corpus, this happened because enabling the whole syntactic parsing function heavily raised the processing time and the RAM usage. To temporarily overcome this solution, I preferred to directly disable the function. This decision was not only tied to hardware limitations, but also to the fact that I wanted to try the capabilities of the LLMs to correctly identify grammatical features without the need of tags to see whether the parsing function could be permanently disabled to free more RAM (even if it would still be better to keep it on so that the model has more explicit data to work better).

3. Data

For testing the tool, I used three custom corpora. The first one is an 88k tokens long corpus created by gathering American/English newspaper articles about the ongoing American/Israeli Iranian crisis in March of 2026. The articles have been gathered by me and other three colleagues, the idea was to collect 25% of the corpus each with four different categories: Left wing, Right wing, Centrist and Economics. Each document has been later marked up with four XML tags: title of the article, author, date and newspaper. This is the main reason that led me to integrate the XML parsing with beautiful soup in the script. This corpus was perfect for testing the tool because of the documents being newspaper articles and the language model (`en_core_web_lg`) being trained exactly on that, and it worked out as a real-life application test for the tool. In total, SpaCy identified 8453 entities, with an entity density of 95.91 entities per 1k tokens and an ETTR of 0.1539. The second corpus is automatically generated with a quick python script, the point of this corpus was not to test the tagging capabilities (in fact it has a TTR of almost 0), but instead the loading time, in fact it works as a stress test it being 18 million tokens long. The last corpus is a much smaller one, only 1471 tokens and a TTR of 0.3812, what differentiate it from the other ones is that I annotated manually the entities in gold standard. It is composed by three AI-generated fake essays made by three imaginary high school students about The Picture of Dorian Gray, the first student did not like it, the second one liked it and the third one did not actually read the book but just a summary on Wikipedia so he has a neutral opinion about it. The point of this was to test the sentiment analysis capabilities of the model and making sure they had variety. All three documents have two XML tags: student name and title of

the essay. This is the most important corpus because it's the one I used for testing the tool, it being small and easy to check for mistakes in the counts or misunderstandings by the LLM. Before continuing with the testing phase, it's important to say that this tool unfortunately only processes .txt files, but it can be modified to also handle other extensions.

4. Testing Phase

4.1 Evaluation of SpaCy NER extraction

I divided the test of the tool in two phases: the statistical and syntactical analysis by SpaCy and the pragmatic analysis. Regarding the first phase I tested it on the gold standard corpus, and the results were good, but not perfect. The data () shows that, as expected, SpaCy tagged with high accuracy overall, even if there are some minor issues like a relatively high number of False Positives and False Negatives (but it is expected since it's a neural network statistical system and not rule-based), yet no instances of completely out of scope mistakes were ever found, so the results are usable overall. It also must be considered that en_core_web_lg has been trained with different kind of texts and was not meant for analyzing this kind of student content, so it is already great results considering the mismatch in topics.

4.2 Evaluation of LLMs capabilities

Testing and establishing whether SpaCy is accurate or not a straightforward process that applies to every kind of statistical model, this because it returns real measurable quantitative output, but that's not the case for really the same for testing LLM responses, so it is necessary to use qualitative methods. I tried all three models, what I found is that Llama3 8B, even if fast, struggles at properly understand questions and respond accordingly to the text, then I tried DeepSeek r-1 8B because it is specifically designed for reasoning, and it was surprisingly the worst performing model, not only had a much longer loading time, he also performed terribly at counting instances tests, severely underestimating frequencies. Then for last I had the possibility to try a larger model thanks to the ARM chip of my pc and the 16gb of unified RAM, which is Qwen 2.5 with 14 billion parameters, and it certainly performed better than both DeepSeek and Llama3, so I decided to stick with it for the official test.

For this I thought it would have been useful to ask different types of questions to test different aspects of the possible outcomes and usages of the tool:

- 1) "Search sentences for each student that explicitly express what they think about the book they read."

The first question tries to test Sentiment Analysis, and Qwen2.5 correctly identified all three students from the XML markups and extracted the sentiment of each student. The most interesting part is that it understood also that the third student was more neutral and did not give any clear opinion about the book beside briefly explaining its moral.

- 2) "In the sentence in which Bobby says that he prefers The Canterville Ghost to The Picture of Dorian Gray, does he have a more enthusiastic or bored sentiment?"

The second question aims at tricking the LLM to hallucinate to test its ability to not create any false information. Also in this case, Qwen2.5 recognized without any problems that the lack of any sentences with meaning that regard the opinion of Bobby towards the The Canterville Ghost,

so accordingly to the instructions given it explicitly refused to give any false information, passing the hallucination test.

- 3) "What year was The Picture of Dorian Gray written? What was the public's initial reaction to the story's release?"

The third question aims at recognizing non pertinent questions. Also in this case, Qwen2.5 followed the given instructions in the script and successfully recognized the questions not pertinent to the contents of the documents, declining the question entirely.

Those were the questions in which, as expected, the LLM performed better, because they are notoriously good at performing tasks like sentiment analysis and distant reading, which by itself is already useful for a pragmatic analysis as long as it's purely qualitative questions regarding the content, which could enhance and speed up the work of researchers to have a prior idea of the contents of the corpus they are querying to avoid spending time searching for information that are not present in the dataset.

After testing the qualitative capabilities of the LLM I then moved on to the most important questions that are also the final objective of the software: mixing qualitative and quantitative querying asking it to count for meaning.

- 4) "Count all Named Entities inside the corpus, return the total number, then make a list of the three most frequent entities per category (Person, Location, Organization and Misc) with raw frequencies."

In this question I tried to test raw instances counting, and this is the point in which Qwen2.5 started to show its most fascinating responses, counting the total instances almost perfectly (68 out of 69 found), but in a completely wrong way, in fact it missed many different types of entities (13 out of 30), but the count is similar almost exact for a different reason: because of the way I structured the tool, under each sentence there's the source with the XML tags, so even if the corpus only had three mentions to the names of the students (John, Mary and Bobby) inside the tags, Qwen counted 23 instances confusing the tags with actual contents of the corpus, bringing the actual count to 43 actual instances. Yet it is important to notice that in some cases the counts are almost precise, like the case of "Dorian Gray" found 18 times (only 2 instances difference from actual 20 present in the corpus).

- 5) "In all the sentences in which Oscar Wilde is cited, count how many of them contain good opinions, bad opinions or neutral opinions."

This last question is the most important one because it aims at testing what would be the central function of the LLM: counting instances of sentences with specific meanings. The response was interesting and slightly confirmed my intuition, in fact, 6 out of the 8 sentences in which Oscar Wild is cited were found, so it's not that precise, yet Qwen perfectly tagged all the six sentences it found with the correct sentiment.

The initial research question was also about why tools like Sketch engine don't integrate natively NER or LLMs, and the answer is clear once you test tools like TrAI_NER with bigger datasets: the loading time increases exponentially, and so does the computing power need. In fact, once I tried TrAI_NER on the other two corpora mentioned, it worked with significantly less efficiency, with longer loading screens and less precise LLM responses. The tool performed better in the Iranian crisis corpus, but it was just 88k token long, when I tried the stress test corpus with 18 million tokens, the loading time raised over 30 minutes just for SpaCy's analysis. So, trying to use these tools on extremely large corpora like on the EnTenTen family would require massive amounts of computing power.

5. Discussion and conclusion

5.1 Analysis of the results

The development and testing of TrAI_NER provided highly valuable insights into the integration of Large Language Models within traditional corpus linguistics workflows. The tool successfully demonstrated that a hybrid architecture—combining the algorithmic precision of SpaCy for distant reading with the pragmatic capabilities of an LLM for close reading—is a viable solution to the limitations found in mainstream tools like Sketch Engine or AntConc. The most fascinating results emerged when trying to combine qualitative and quantitative queries: When Qwen2.5 was tasked with counting entities, it suffered from a phenomenon known as *Metadata Confusion* (or *Prompt Distraction*). Rather than scanning the textual content of the essays, the model erroneously counted the occurrences of the students' names hidden within the XML metadata tags passed in the prompt, entirely missing secondary characters like Hyde or Jekyll. This hallucination proves that LLMs flatten the structural context, making them virtually unusable for frequency counting, thereby validating the necessity of TrAI_NER's hybrid approach where counting is strictly delegated to the NER pipeline. Conversely, when tasked with evaluating sentiment, detecting sarcasm, or avoiding out-of-domain hallucinations, the LLM performed exceptionally well, proving its worth as a qualitative analysis assistant, and the paradox of a model that correctly identifies meaning but can't count the instances most certainly relies in one fundamental incompatibility with the task: the LLM did not count wrong, in fact it didn't count at all, it just statistically guessed the results, getting close to it, but the outputs are far from being accurate or employable in any research or academic context. The most efficient use of TrAI_NER would therefore be to exploit the accurate auto-tagging of SpaCy and using the LLM for strictly qualitative and pragmatic distant reading, and to not trust any frequencies that emerge from the outputs.

5.2 Final considerations about TrAI_NER

Having explained the results, it is possible to make some considerations on the concept of the tool: first, it's clear that it would make sense to not have it as a standalone tool, but instead to have it integrated inside other more powerful, cloud-based Engines. This would allow to scale with larger LLMs and increase the overall performance. Another problem of keeping it as it is, is that many important information are left behind disabling the syntactical parsing, therefore using cloud computing would allow to keep the parsing enabled and exploit more detailed linguistic information about the corpus. The last and more important feature that is missing and severely limits the querying power of the tool is CQL (and KWIC broadly) that would filter much more the output and drastically reduce the noise for the LLM.

References

- AI@Meta (2024). The Llama 3 Herd of Models. *arXiv preprint arXiv:2407.21783*.
- Anthony, L. (2022). *AntConc* (Version 4.2.0) [Computer Software]. Waseda University.
- DeepSeek-AI (2025). DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. *arXiv preprint arXiv:2501.12948*.
- Dziri, S., Lu, X., Sclar, M., et al. (2023). Faith and Fate: Limits of Transformers on Compositionality. *Advances in Neural Information Processing Systems*, 36.

Harris, C. R., Millman, K. J., van der Walt, S. J., et al. (2020). Array programming with NumPy. *Nature*, 585(7825), 357-362.

Honnibal, M., & Montani, I. (2017). *SpaCy 2: Natural language understanding with Bloom embeddings, convolutional neural networks and incremental parsing*.

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90-95.

Kilgarriff, A., Rychlý, P., Smrž, P., & Tugwell, D. (2004). The Sketch Engine. *EURALEX 2004 Proceedings*, 105-116.

McKinney, W. (2010). Data Structures for Statistical Computing in Python. In *Proceedings of the 9th Python in Science Conference* (pp. 56-61).

Mueller, A. C. (2023). word_cloud [Computer Software]. GitHub. https://github.com/amueller/word_cloud

Ollama. (2024). *Ollama: Get up and running with large language models locally* [Computer Software]. <https://ollama.com>

Qwen Team (2024). Qwen2.5 Technical Report. *arXiv preprint arXiv:2412.15221*.

Razeghi, Y., Logan IV, R. L., Gardner, M., & Singh, S. (2022). Impact of Pretraining Term Frequencies on Few-Shot Reasoning. *Findings of the Association for Computational Linguistics: EMNLP 2022*.

Richardson, L. (2024). *Beautiful Soup* (Version 4.12) [Computer Software]. <https://www.crummy.com/software/BeautifulSoup/>

Streamlit Inc. (2024). *Streamlit* [Computer Software]. <https://streamlit.io>

Waskom, M. L. (2021). seaborn: statistical data visualization. *Journal of Open-Source Software*, 6(60), 3021.

Appendix

Table 1. Frequencies and accuracy of SpaCy auto-tagging.

Category	Entity	G.S. Frequency	Raw Frequency	TP	FP	FN
Person	Dorian	18	16	16	0	2
Person	Dorian Gray	2	7	2	5	0
Person	Oscar Wilde	7	6	6	0	1
Person	Basil Hallward	3	3	3	0	0
Person	Henry Wotton	3	3	3	0	0
Location	London	4	3	3	0	1
Person	Hyde	2	3	2	1	0

Person	Sibyl	2	2	2	0	0
Person	Basil	2	2	2	0	0
Person	Juliet	1	2	1	1	0
Person	Henry	2	2	2	0	0
Person	Jekyll	2	2	2	0	0
Person	Wilde	1	1	1	0	0
Person	Stevenson	1	1	1	0	0
Person	Danvers Carew	1	1	1	0	0
Person	Shakespeare	1	1	1	0	0
Person	Robert Louis Stevenson	1	1	1	0	0
Person	James Vane	1	1	1	0	0
Location	the East End	1	1	1	0	0
Person	Sibyl Vane	3	2	2	0	1
MISC	The Picture of Dorian Gray	5	0	0	0	5
MISC	Romeo and Juliet	1	0	0	0	1
MISC	The Strange Case of Dr. Jekyll and Mr. Hyde	1	0	0	0	1
MISC	The Strange Case of Dr. Jekyll	0	1	0	1	0
MISC	Victorian	3	0	0	0	3
Person	Romeo	0	1	0	1	0
Person	Oscar Wilde's	0	1	0	1	0
Person	Sibyl Vane's	0	1	0	1	0
Location	Victorian London	0	1	0	1	0
TOTAL		68	65	53	12	15

Table 2. Precision, Recall and F1-score for gold standard corpus

Precision	Recall	F1-Score
81.54%	77.94%	79.70%

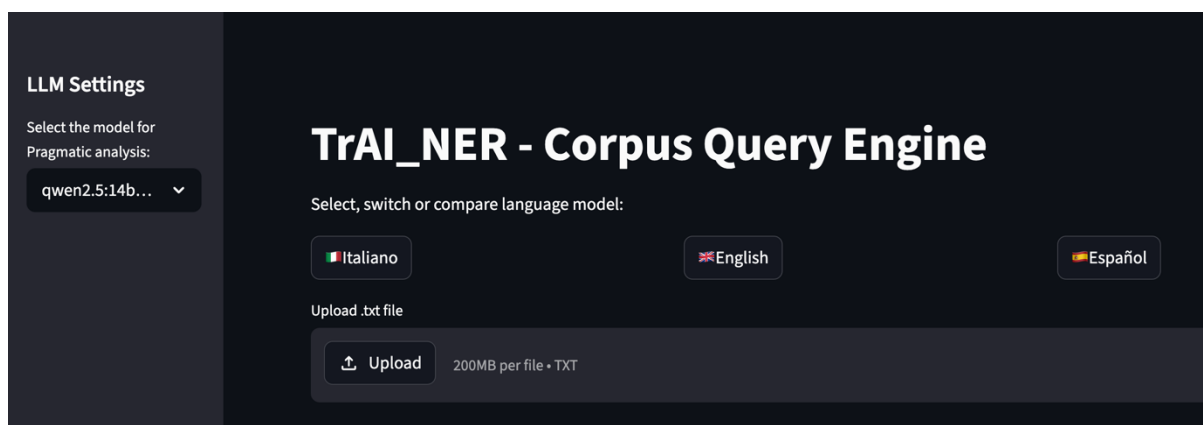


Image 1. Starting the engine.

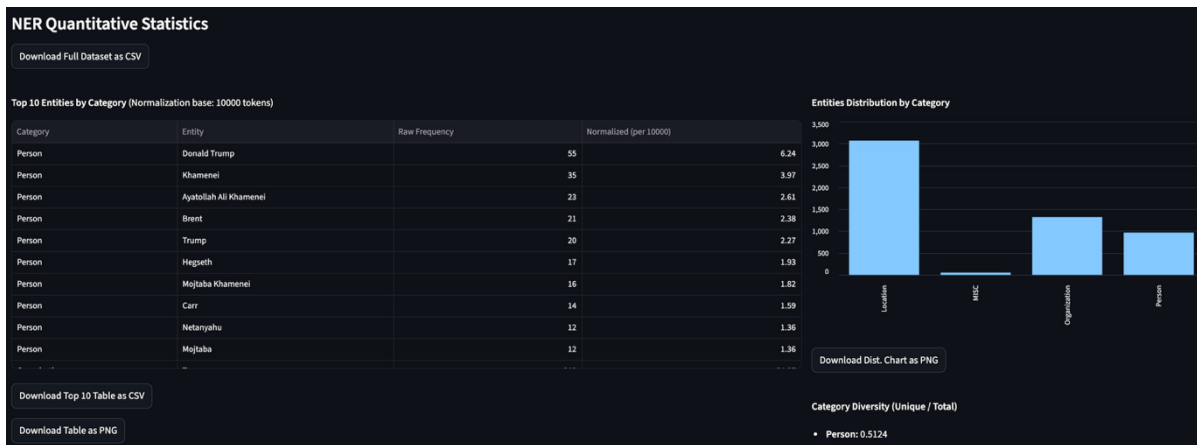


Image 2. Graphical quantitative statistics section.

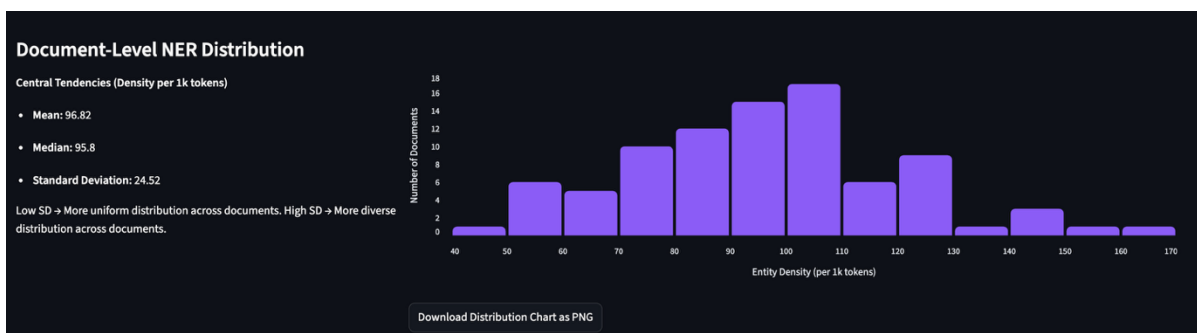


Image 3. Document-level NER standard deviation and entity distribution.

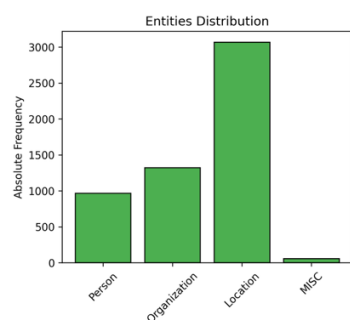


Image 4. Example of PNG graph downloaded.

Explore per NER category

All Person Organization Location MISC

2302 sentences with any NER tag.

Filter in All:

Showing results 1 to 100 of 2302:

1. A slow-burning economic crisis in **Iran** reached a tipping point in late December after a slump in the national currency to a record low made even basic goods unaffordable for much of the population. (Source: date: 2026-01-14, author: Arsalan Shahla, newspaper: Bloomberg, title: How Iran Sanctions and a Currency Crash Triggered Mass Protests)

2. Shopkeepers and merchants were among the first to react, taking to the streets of the capital **Tehran** to vent their frustration at a government they accused of mismanaging the economy. (Source: date: 2026-01-14, author: Arsalan Shahla, newspaper: Bloomberg, title: How Iran Sanctions and a Currency Crash Triggered Mass Protests)

Pragmatic Analysis (qwen2.5:14b)

Type here and press Enter to send...

Image 5. LLM interaction interface

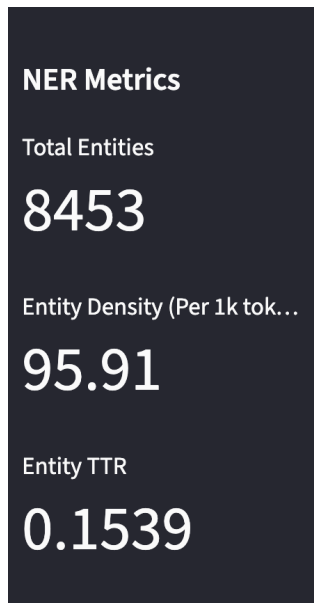


Image 6. NER metrics in general statistics left menu.